

Configuracion de Shell (.bashrc)

NVIDIA Jetson AGX Orin 64 GB Developer Kit

Componente	Detalle
Hardware	NVIDIA Jetson AGX Orin Developer Kit 64 GB
Sistema operativo	Ubuntu 22.04.5 LTS (aarch64)
Kernel	5.15.185-tegra
CPU	ARMv8 rev 1 (v8l) — 12 nucleos @ 2.2 GHz
RAM	64 GB LPDDR5 (unificada CPU/GPU)
JetPack	6.2.2 (nvidia-l4t-core 36.5.0)
CUDA	12.6 (V12.6.68)
cuDNN	9.3.0
TensorRT	10.3.0.30
OpenCV	4.8.0

Tutorial paso a paso para configurar un entorno de desarrollo de IA optimizado mediante la personalizacion del archivo .bashrc. Incluye rutas CUDA, variables de entorno, alias de rendimiento, Docker GPU, Ollama, pyenv, NVM y mejores practicas.

Marzo 2026 | v1.0

Tabla de Contenidos

1. Fundamentos: Que es .bashrc y como funciona
2. Prerequisitos y seguridad
3. Configuracion de rutas CUDA 12.6
4. Bibliotecas del sistema: TensorRT, cuDNN y Tegra
5. Binarios locales del usuario
6. Directorio de modelos de IA
7. Variables de Hugging Face
8. Flags de compilacion paralela
9. Configuracion de Ollama
10. Entornos virtuales de Python y pyenv
11. NVM para Node.js (con carga diferida)
12. Atajos de rendimiento del Jetson
13. Aliases de monitoreo del sistema
14. Aliases de Docker GPU
15. Aliases del sistema de archivos
16. Aliases de sistema y mantenimiento
17. Aliases de Git
18. Optimizacion del historial de Bash
19. Deduplicacion del PATH
20. Saludo de bienvenida del Jetson
21. Bloque completo de produccion
22. Aplicar y verificar
23. Hugging Face Token (Opcional)
24. Cache en NVMe (Opcional)
25. Resolucion de problemas
26. Fuentes y referencias

1. Fundamentos: Que es .bashrc y como funciona

El archivo `~/ .bashrc` es un script de shell que Bash ejecuta automaticamente cada vez que abres una nueva terminal interactiva (no de inicio de sesion). Es el lugar principal donde personalizas tu entorno de linea de comandos: variables de entorno, alias, funciones, prompt y mas.

Flujo de ejecucion de Bash

Cuando Bash inicia, sigue una secuencia especifica para cargar archivos de configuracion, dependiendo del tipo de shell:

Tipo de Shell	Archivos que carga	Ejemplo comun
Login interactivo (SSH, TTY)	<code>/etc/profile</code> luego <code>~/ .bash_profile</code> o <code>~/ .bash_login</code> o <code>~/ .profile</code>	Conexion SSH al Jetson
No-login interactivo (terminal local)	<code>/etc/bash.bashrc</code> luego <code>~/ .bashrc</code>	Abrir terminal con <code>Ctrl+Alt+T</code>
No interactivo (scripts)	Solo <code>\$BASH_ENV</code> si esta definido	Ejecutar un script <code>.sh</code>

INFO

En Ubuntu 22.04, el archivo `~/ .profile` ya contiene codigo que carga `~/ .bashrc` automaticamente. Esto significa que tus configuraciones de `.bashrc` se aplican tanto en shells de login como en shells no-login interactivos.

Por que es importante en Jetson

En un Jetson AGX Orin, necesitas que cada terminal nueva tenga acceso inmediato a:

- CUDA 12.6 — compilador `nvcc` y bibliotecas de GPU
- TensorRT / cuDNN — aceleracion de inferencia
- Python con entornos virtuales — aislamiento de dependencias
- Docker con GPU — contenedores con acceso a la GPU
- Ollama — servidor local de modelos de lenguaje
- Herramientas de monitoreo — `tegrastats`, `nvpmodel`, temperaturas

Sin una configuracion correcta del `.bashrc`, cada vez que abras una terminal tendrias que escribir manualmente las rutas y variables. El `.bashrc` automatiza todo esto.

2. Prerequisitos y seguridad

Paso 1: Abrir una terminal

Presiona Ctrl + Alt + T en tu Jetson para abrir la terminal, o busca "Terminal" en el menu de aplicaciones.

Paso 2: Hacer una copia de seguridad del .bashrc actual

Antes de modificar cualquier cosa, crea una copia de respaldo. Si algo sale mal, podras restaurar el archivo original.

Terminal

```
cp ~/.bashrc ~/.bashrc.backup.$(date +%Y%m%d)
```

Este comando crea un archivo como ~/.bashrc.backup.20260331 con la fecha actual. Si necesitas restaurar:

Restaurar backup

```
cp ~/.bashrc.backup.20260331 ~/.bashrc  
source ~/.bashrc
```

IMPORTANTE

Nunca edites .bashrc directamente sin tener un backup. Un error de sintaxis puede impedir que tu terminal funcione correctamente.

Paso 3: Verificar que Bash es tu shell activo

Terminal

```
echo $SHELL  
# Resultado esperado: /bin/bash
```

Paso 4: Abrir el archivo para edicion

Usaremos nano, un editor de texto sencillo incluido en Ubuntu:

Terminal

```
nano ~/.bashrc
```

INFO

En nano: Ctrl+O para guardar, Ctrl+X para salir. Usa las flechas del teclado para navegar. Ctrl+End lleva al final del archivo.

Estrategia: Agregar un bloque limpio al final

En lugar de editar línea por línea, pegaremos un bloque completo al final del `.bashrc`. Esto es mas seguro y organizado. Los guardas al inicio del bloque previenen entradas duplicadas.

3. Configuracion de rutas CUDA 12.6

CUDA (Compute Unified Device Architecture) es el toolkit de NVIDIA para programacion de GPU. Tu Jetson tiene CUDA 12.6 instalado con JetPack 6.2.2, pero el sistema necesita saber donde encontrar sus binarios y bibliotecas.

Que hace cada variable

Variable	Proposito	Ruta en Jetson
CUDA_HOME	Directorio raiz de CUDA. Usado por CMake, PyTorch, y otros frameworks.	/usr/local/cuda-12.6
PATH	Agrega nvcc y otros binarios de CUDA al PATH del sistema.	/usr/local/cuda-12.6/bin
LD_LIBRARY_PATH	Indica al enlazador dinamico donde buscar bibliotecas .so de CUDA.	/usr/local/cuda-12.6/lib64 /usr/local/cuda-12.6/extras/CUPTI/lib64

Codigo para agregar al .bashrc

```
~/bashrc - Seccion CUDA

# --- CUDA 12.6 ---
export CUDA_HOME=/usr/local/cuda-12.6
export PATH=/usr/local/cuda-12.6/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda-12.6/lib64:/usr/local/cuda-12.6/extras/CUPTI/lib64:
```

Que es CUPTI y por que incluirlo

CUPTI (CUDA Profiling Tools Interface) es la biblioteca de perfilado de CUDA. Herramientas como nvprof, nsight-sys y PyTorch Profiler la necesitan. Sin esta ruta en LD_LIBRARY_PATH, recibiras advertencias al intentar perfilar tu codigo CUDA.

Verificacion

Terminal – Verificación CUDA

```
# Despues de aplicar cambios (source ~/.bashrc):  
echo $CUDA_HOME  
# -> /usr/local/cuda-12.6  
  
nvcc --version  
# -> Cuda compilation tools, release 12.6, V12.6.68  
  
ls /usr/local/cuda-12.6/extras/CUPTI/lib64/  
# -> libcupti.so  libcupti.so.12  ...
```

4. Bibliotecas del sistema: TensorRT, cuDNN y Tegra

En JetPack 6.2.2, TensorRT, cuDNN y las bibliotecas Tegra se instalan en rutas del sistema `/usr/lib/aarch64-linux-gnu/`. Necesitamos asegurar que el enlazador dinamico las encuentre, especialmente las bibliotecas Tegra que contienen drivers de GPU propietarios de NVIDIA.

Rutas importantes

Biblioteca	Ruta de includes (.h)	Ruta de libs (.so)
TensorRT 10.3.0	<code>/usr/include/aarch64-linux-gnu/</code>	<code>/usr/lib/aarch64-linux-gnu/</code>
cuDNN 9.3.0	<code>/usr/include/aarch64-linux-gnu/</code>	<code>/usr/lib/aarch64-linux-gnu/</code>
Tegra (drivers GPU)	N/A	<code>/usr/lib/aarch64-linux-gnu/tegra/</code>

Codigo para agregar al .bashrc

```
~/ .bashrc — Seccion TensorRT/Tegra

# --- TensorRT / Tegra (bindings de Python y drivers GPU) ---
export LD_LIBRARY_PATH=/usr/lib/aarch64-linux-gnu/tegra:$LD_LIBRARY_PATH
```

INFO

Las bibliotecas en `/usr/lib/aarch64-linux-gnu/` generalmente se encuentran automaticamente por el enlazador. La ruta `/tegra/` es la que necesita declaracion explicita porque contiene los drivers propietarios de GPU especificos del Jetson.

Verificacion

```
Terminal — Verificacion de bibliotecas

# Verificar TensorRT
dpkg -l | grep tensorrt | head -3
# -> tensorrt 10.3.0.30-1+cuda12.5 arm64

# Verificar cuDNN
dpkg -l | grep cudnn | head -3

# Verificar bibliotecas Tegra
ls /usr/lib/aarch64-linux-gnu/tegra/ | head -5
# -> libnvidia-encode.so.1 libnvidia-ml.so.1 ...
```


5. Binarios locales del usuario

Cuando instalas paquetes de Python con `pip install --user`, los binarios ejecutables se colocan en `~/.local/bin/`. Sin esta ruta en el PATH, herramientas como `huggingface-cli`, `jupyter`, o `tensorboard` no serán encontradas.

```
~/.bashrc — Binarios locales
```

```
# --- Binarios locales del usuario (pip install --user) ---  
export PATH="$HOME/.local/bin:$PATH"
```

TIP

Colocar `$HOME/.local/bin` al INICIO del PATH (antes de `$PATH`) permite que tus versiones locales tengan prioridad sobre las del sistema.

6. Directorio de modelos de IA

Definir una variable de entorno para la ubicación de tus modelos facilita la organización y permite que scripts y notebooks la referencien sin rutas absolutas fijas.

```
~/.bashrc — Directorio de modelos
```

```
# --- Directorio de modelos (cambiar si usas otra ubicación) ---  
export MODELS_DIR="$HOME/models"
```

Crea el directorio si no existe:

```
Terminal
```

```
mkdir -p ~/models
```

TIP

Si tienes un disco NVMe montado (por ejemplo en `/data`), cambia esta ruta a `/data/models` para tener más espacio y mejor velocidad de lectura.

7. Variables de Hugging Face

Hugging Face Hub es el repositorio principal de modelos de IA (Llama, Mistral, Qwen, etc.). Estas variables controlan cómo se descargan y almacenan los modelos.

Variable	Valor	Descripcion
HF_HUB_ENABLE_HF_TRANSFER	1	Habilita hf_transfer para descargas mas rapidas. Usa multiples conexiones paralelas.
HF_HOME	\$HOME/.cache/huggingface	Directorio donde se almacenan modelos en cache (puede ser 10-100 GB).

```
~/.bashrc — Hugging Face
```

```
# --- Hugging Face: descargas aceleradas ---  
export HF_HUB_ENABLE_HF_TRANSFER=1  
export HF_HOME="$HOME/.cache/huggingface"
```

ADVERTENCIA

hf_transfer es mas rapido solo cuando tienes buena conexion de red. En conexiones lentas puede no aportar beneficio o incluso ser mas lento. Si notas problemas, cambia el valor a 0.

8. Flags de compilacion paralela

Tu Jetson AGX Orin tiene 12 nucleos de CPU. Al compilar software desde codigo fuente (make, cmake), podemos usar todos los nucleos simultaneamente para acelerar el proceso significativamente.

```
~/bashrc — Flags de compilacion
```

```
# --- Build flags: usar todos los nucleos ---
export MAKEFLAGS="-j$(nproc)"
export CMAKE_BUILD_PARALLEL_LEVEL=$(nproc)
```

`$(nproc)` devuelve el numero de nucleos disponibles (12 en tu caso). Esto hace que:

- **MAKEFLAGS:** Pasa `-j12` automaticamente a `make`
- **CMAKE_BUILD_PARALLEL_LEVEL:** Configura CMake para compilacion paralela

Verificacion

```
Terminal
```

```
echo "Nucleos para compilacion: $MAKEFLAGS"
# -> Nucleos para compilacion: -j12

echo "Nucleos cmake: $CMAKE_BUILD_PARALLEL_LEVEL"
# -> Nucleos cmake: 12
```

9. Configuracion de Ollama

Ollama es un servidor local de modelos de lenguaje (LLMs) que usa llama.cpp con aceleracion CUDA. Estas variables son utiles cuando Ollama se ejecuta como servicio nativo (no Docker) o cuando necesitas que scripts de Python se conecten.

```
~/bashrc — Ollama
```

```
# --- Ollama (si se ejecuta como servicio nativo) ---
export OLLAMA_HOST="http://localhost:11434"
export OLLAMA_MODELS="$HOME/.ollama/models"
```

Variable	Valor por defecto	Descripcion
OLLAMA_HOST	127.0.0.1:11434	Direccion del servidor. Cambiar a 0.0.0.0 para acceso por red local.
OLLAMA_MODELS	~/ollama/models	Ubicacion de modelos. Mover a NVMe si hay espacio disponible.

Variable	Valor por defecto	Descripcion
OLLAMA_KEEP_ALIVE	5m	Tiempo que un modelo permanece en memoria desde su ultimo uso.
OLLAMA_FLASH_ATTENTION	desactivado	Habilitar flash attention para mayor velocidad de inferencia.

INFO

Si Ollama corre como servicio systemd (la instalacion por defecto en Linux), las variables de entorno deben configurarse con 'systemctl edit ollama.service' en la seccion [Service], no en el .bashrc. Las variables del .bashrc son utiles para el cliente CLI y scripts.

10. Entornos virtuales de Python y pyenv

El sistema Jetson usa Python 3.10 del sistema. Para proyectos de IA, es fundamental usar entornos virtuales que aislen las dependencias de cada proyecto sin afectar al sistema.

Aliases rapidos para entornos virtuales

~/bashrc — Aliases de Python

```
# --- Helpers para entornos virtuales de Python ---
# Activar el entorno principal de IA: escribir "ai" en terminal
alias ai='source ~/envs/ai_env/bin/activate'
# Desactivar cualquier entorno: escribir "da"
alias da='deactivate'
```

Para crear el entorno virtual por primera vez:

Terminal — Crear entorno virtual

```
mkdir -p ~/envs
python3 -m venv ~/envs/ai_env
source ~/envs/ai_env/bin/activate
pip install --upgrade pip
```

pyenv (si esta instalado)

pyenv permite instalar y gestionar multiples versiones de Python. La siguiente configuracion lo inicializa automaticamente si esta instalado, sin causar errores si no lo esta.

~/bashrc — pyenv

```
# --- pyenv (si esta instalado) ---
export PYENV_ROOT="$HOME/.pyenv"
[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"
command -v pyenv > /dev/null && eval "$(pyenv init - bash)" && eval "$(pyenv virtualenv-in
```

Linea por linea:

- Linea 1: Define donde esta instalado pyenv
- Linea 2: Solo agrega pyenv al PATH si el directorio existe
- Linea 3: Solo inicializa pyenv si el comando esta disponible

TIP

La ventaja de pyenv local es que crea un archivo .python-version en el directorio del proyecto, activando automaticamente la version correcta cuando entras al directorio.

11. NVM para Node.js (con carga diferida)

NVM (Node Version Manager) puede hacer que tu terminal tarde 1-3 segundos adicionales en abrir porque carga todo su entorno al iniciar. La solución es la carga diferida (lazy loading): NVM solo se carga cuando realmente usas node, npm o nvm por primera vez.

Opcion A: Carga estandar (mas lenta)

~/bashrc — NVM estandar

```
# --- NVM estandar (agrega ~1-3s al inicio de cada terminal) ---
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"
```

Opcion B: Carga diferida (recomendada)

~/bashrc — NVM lazy load (recomendado)

```
# --- NVM con carga diferida (inicio instantaneo) ---
export NVM_DIR="$HOME/.nvm"
if [ -s "$NVM_DIR/nvm.sh" ]; then
    lazynvm() {
        unset -f nvm node npm npx
        [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
        [ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"
    }
    nvm() { lazynvm; nvm "$@"; }
    node() { lazynvm; node "$@"; }
    npm() { lazynvm; npm "$@"; }
    npx() { lazynvm; npx "$@"; }
fi
```

Con la carga diferida, la terminal abre instantaneamente. La primera vez que ejecutes node, npm o nvm, se cargara NVM automaticamente (solo esa vez).

12. Atajos de rendimiento del Jetson

El Jetson AGX Orin tiene multiples modos de potencia (nvpmodel) que controlan cuantos nucleos de CPU estan activos, las frecuencias de CPU/GPU/EMC y el consumo electrico. Estos alias te permiten cambiar entre modos con un solo comando.

```
~/bashrc - Rendimiento

# --- Atajos de rendimiento del Jetson ---
alias maxn='sudo nvpmodel -m 0 && sudo jetson_clocks && echo "MAXN mode + clocks locked"'
alias powersave='sudo nvpmodel -m 3 && echo "15W power-save mode"'
alias perf='sudo nvpmodel -q && sudo jetson_clocks --show 2>/dev/null | grep -E "(CPU|GPU|'
```

Alias	Que hace	Cuando usarlo
maxn	Modo MAXN (maximo rendimiento) + fija relojes al maximo.	Inferencia de modelos, compilacion, benchmarks. Consumo: ~60W.
powersave	Modo de ahorro de energia (15W).	Tareas ligeras, cuando no necesitas GPU. Reduccion de calor.
perf	Muestra el modo actual y frecuencias de CPU/GPU/EMC.	Verificar en que modo esta tu Jetson antes de una tarea.

Modos de potencia del AGX Orin 64 GB

Modo	Potencia	CPUs activas	GPU Max	Uso tipico
0 (MAXN)	~60W	12 @ 2.2 GHz	1.3 GHz	Inferencia / compilacion
1 (50W)	50W	12 @ 2.0 GHz	1.3 GHz	Carga moderada
2 (30W)	30W	8 @ 1.5 GHz	930 MHz	Carga ligera
3 (15W)	15W	4 @ 1.0 GHz	520 MHz	Ahorro de energia

ADVERTENCIA

jetson_clocks fija los relojes al maximo y desactiva el escalado dinamico de frecuencia (DVFS). Su efecto NO persiste tras un reinicio. Si deseas rendimiento maximo persistente, anade el comando a un servicio systemd.

13. Alias de monitoreo del sistema

Estos alias proporcionan acceso rapido a las herramientas de monitoreo especificas del Jetson.

```
~/bashrc — Monitoreo

# --- Monitoreo del Jetson ---
alias stats='tegrastats --interval 1000'
alias temps='cat /sys/class/thermal/thermal_zone*/temp 2>/dev/null | awk "{printf \"\%.1f
alias memfree='free -h | grep -E \"^(Mem|Swap)\"'
alias gpucheck='tegrastats | head -1'
```

Alias	Descripcion	Ejemplo de salida
stats	tegrastats continuo (CPU, GPU, RAM, temp, potencia) cada 1s.	RAM 7467/62827MB CPU [5%@2201,...] GR3D 0%@1300
temps	Todas las temperaturas de los sensores termicos en grados Celsius.	42.5 C 45.0 C 38.7 C
memfree	Uso de RAM y SWAP en formato legible.	Mem: 7.3Gi/61Gi Swap: 0B/31Gi
gpucheck	Instantanea unica de tegrastats.	Una linea con todos los metrics actuales

14. Aliases de Docker GPU

En Jetson con JetPack 6.2, Docker se configura con nvidia-container-runtime como runtime por defecto. Estos aliases simplifican la ejecucion de contenedores con acceso a la GPU.

```
~/ .bashrc — Docker

# --- Docker GPU ---
alias drun='docker run --runtime nvidia --gpus all'
alias ollama='docker exec -it ollama ollama'
```

drun — Ejecuta cualquier contenedor Docker con acceso a la GPU del Jetson. Ejemplo de uso:

```
Terminal — Uso de drun

# Ejecutar un contenedor con GPU:
drun -it --rm nvcr.io/nvidia/l4t-pytorch:r36.4.0-pt2.5-py3 bash

# Equivalente completo sin alias:
docker run --runtime nvidia --gpus all -it --rm ...
```

ollama — Si Ollama corre en Docker, este alias permite ejecutar comandos directamente sin escribir docker exec:

```
Terminal — Uso del alias ollama

# Con alias:
ollama list
ollama run qwen2.5:7b

# Sin alias (equivalente):
docker exec -it ollama ollama list
```

ADVERTENCIA

Si Ollama esta instalado nativamente (no en Docker), elimina el alias 'ollama' del .bashrc. El alias sobrescribiria el comando nativo y causaria errores.

15. Aliases del sistema de archivos

```
~/bashrc — Sistema de archivos

# --- Sistema de archivos ---
alias ls='ls --color=auto'
alias ll='ls -alFh --color=auto'
alias la='ls -A --color=auto'
alias ..='cd ..'
alias ...='cd ../..'
alias mkdir='mkdir -pv'
```

Alias	Que hace
ls	Lista archivos con colores (mas facil de leer).
ll	Lista detallada: permisos, tamano legible, tipo.
la	Lista incluyendo archivos ocultos (los que empiezan con punto).
..	Subir un nivel en el directorio.
...	Subir dos niveles.
mkdir	Crea directorios incluyendo padres (-p) y muestra lo creado (-v).

16. Aliases de sistema y mantenimiento

```
~/bashrc — Sistema

# --- Sistema ---
alias update='sudo apt update && sudo apt upgrade -y && sudo apt autoremove -y'
alias disk='df -h | grep -E "^(/dev|Filesystem)"'
alias ports='ss -tlnp'
```

- update — Un solo comando para actualizar todo el sistema: busca actualizaciones, las instala y limpia paquetes huérfanos.
- disk — Muestra solo las particiones relevantes de disco, filtrando ruido de montajes temporales.
- ports — Lista todos los puertos en escucha (TCP y UDP). Util para verificar que servicios como Ollama (11434), n8n (5678) o Jupyter (8888) están activos.

17. Aliases de Git

```
~/bashrc - Git

# --- Git ---
alias gs='git status'
alias gd='git diff'
alias gl='git log --oneline -10'
```

Estos aliases ahorran tiempo con los comandos de Git mas frecuentes. gs para ver el estado, gd para ver diferencias, gl para los ultimos 10 commits.

18. Optimizacion del historial de Bash

Por defecto, Bash guarda solo 500-1000 comandos y puede perder historial al tener multiples terminales abiertas. Esta configuracion lo mejora significativamente.

```
~/bashrc - Historial

# --- Historial optimizado ---
HISTCONTROL=ignoreboth      # Ignora duplicados y comandos con espacio
HISTSIZE=100000             # Comandos en memoria
HISTFILESIZE=10000000       # Comandos en archivo
HISTIGNORE="ls:ll:cd:pwd:bg:fg:history" # Ignora comandos triviales
shopt -s histappend         # Append en vez de sobrescribir
PROMPT_COMMAND="history -a; history -n; $PROMPT_COMMAND"
```

Opcion	Efecto
HISTCONTROL=ignoreboth	Combina ignoredups (no duplicados consecutivos) e ignorespace (comandos con espacio al inicio no se guardan, util para passwords).
HISTSIZE=100000	Mantiene hasta 100,000 comandos en memoria (200x mas que el default de 500).
shopt -s histappend	Agrega al archivo de historial en vez de sobrescribirlo. Evita perder historial cuando hay varias terminales.
history -a; history -n	Escribe al archivo inmediatamente (-a) y recarga entradas nuevas de otras terminales (-n). Mas eficiente que 'history -a; history -c; history -r'.

TIP

Con HISTCONTROL=ignorespace, puedes ejecutar comandos sensibles (como exportar tokens) sin que queden en el historial: solo agrega un espacio antes del comando. Ejemplo: ' export HF_TOKEN=hf_xxx'

19. Deduplicacion del PATH

Cada vez que ejecutas `source ~/.bashrc`, las lineas `export PATH=...:$PATH` agregan las mismas rutas al PATH. Sin deduplicacion, tu PATH crece indefinidamente, haciendo que la busqueda de comandos sea mas lenta.

```
~/.bashrc — Deduplicacion PATH
```

```
# --- Deduplicar PATH (previene crecimiento en recargas) ---
export PATH=$(echo -n "$PATH" | awk -v RS=: -v ORS=: '!x[$0]++' | sed 's:/:$/:')
```

Este one-liner usa `awk` para eliminar entradas duplicadas manteniendo el orden original, y `sed` para limpiar el separador final.

Verificacion

```
Terminal — Verificar deduplicacion
```

```
# Antes de deduplicar (ejemplo con duplicados):
echo $PATH | tr ":" "\n" | sort | uniq -c | sort -rn | head -5
#      3 /usr/local/cuda-12.6/bin      <- duplicado!
#      1 /usr/bin

# Despues de source ~/.bashrc con deduplicacion:
echo $PATH | tr ":" "\n" | sort | uniq -c | sort -rn | head -5
#      1 /usr/local/cuda-12.6/bin      <- limpio!
```

20. Saludo de bienvenida del Jetson

Este bloque muestra informacion basica del Jetson cada vez que abres una terminal: modo de potencia actual y uso de RAM.

```
~/.bashrc — Saludo de bienvenida
```

```
# --- Saludo al abrir terminal ---
if command -v tegrastats &>/dev/null; then
    POWER_MODE=$(sudo nvpmode1 -q 2>/dev/null | grep "NV Power Mode" | awk '{print $NF}')
    echo "Jetson AGX Orin 64GB | Mode: ${POWER_MODE:-unknown} | $(free -h | grep Mem | awk
fi
```

Ejemplo de salida al abrir una terminal:

```
Terminal
```

```
Jetson AGX Orin 64GB | Mode: MAXN | RAM: 7.3Gi/61Gi
```

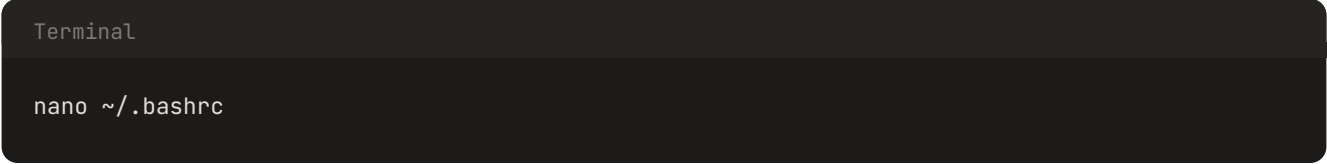
ADVERTENCIA

El comando 'sudo nvpmode1 -q' puede solicitar contrasena al abrir la terminal. Si esto te molesta, elimina estas lineas o configura sudo sin password para nvpmode1 con: `echo "$USER ALL=(ALL) NOPASSWD: /usr/sbin/nvpmode1" | sudo tee /etc/sudoers.d/nvpmode1`

21. Bloque completo de produccion

A continuacion se presenta el bloque completo para pegar al final de tu ~/.bashrc. Incluye todas las secciones explicadas anteriormente en un solo bloque coherente.

Paso 1: Abrir el archivo

A terminal window with a dark background. The title bar says "Terminal". The command "nano ~/.bashrc" is entered at the prompt.

```
Terminal
```

```
nano ~/.bashrc
```

Paso 2: Ir al final del archivo

Presiona Ctrl + End en nano para ir al final del archivo.

Paso 3: Pegar el siguiente bloque completo

```
~/bashrc — BLOQUE COMPLETO (parte 1 de 3)
```

```
# =====
# Jetson AGX Orin — AI Development Environment
# JetPack 6.2.2 | CUDA 12.6 | aarch64
# =====

# --- Guard: solo ejecutar en shells interactivos ---
[[ $- != *i* ]] && return

# --- CUDA 12.6 ---
export CUDA_HOME=/usr/local/cuda-12.6
export PATH=/usr/local/cuda-12.6/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda-12.6/lib64:/usr/local/cuda-12.6/extras/CUPTI/lib64:

# --- TensorRT / Tegra (drivers GPU) ---
export LD_LIBRARY_PATH=/usr/lib/aarch64-linux-gnu/tegra:$LD_LIBRARY_PATH

# --- Binarios locales (pip install --user) ---
export PATH="$HOME/.local/bin:$PATH"

# --- Directorio de modelos ---
export MODELS_DIR="$HOME/models"

# --- Hugging Face ---
export HF_HUB_ENABLE_HF_TRANSFER=1
export HF_HOME="$HOME/.cache/huggingface"

# --- Compilacion paralela (12 nucleos) ---
export MAKEFLAGS="-j$(nproc)"
export CMAKE_BUILD_PARALLEL_LEVEL=$(nproc)

# --- Ollama (servicio nativo o cliente) ---
export OLLAMA_HOST="http://localhost:11434"
export OLLAMA_MODELS="$HOME/.ollama/models"

# --- Entornos virtuales de Python ---
alias ai='source ~/envs/ai_env/bin/activate'
alias da='deactivate'
```

~/ .bashrc — BLOQUE COMPLETO (parte 2 de 3)

```
# --- pyenv (si esta instalado) ---
export PYENV_ROOT="$HOME/.pyenv"
[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"
command -v pyenv > /dev/null && eval "$(pyenv init - bash)" && eval "$(pyenv virtualenv-in-

# --- NVM con carga diferida ---
export NVM_DIR="$HOME/.nvm"
if [ -s "$NVM_DIR/nvm.sh" ]; then
    lazynvm() {
        unset -f nvm node npm npx
        [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
        [ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"
    }
    nvm() { lazynvm; nvm "$@"; }
    node() { lazynvm; node "$@"; }
    npm() { lazynvm; npm "$@"; }
    npx() { lazynvm; npx "$@"; }
fi

# --- Rendimiento del Jetson ---
alias maxn='sudo nvpmodel -m 0 && sudo jetson_clocks && echo "MAXN mode + clocks locked"'
alias powersave='sudo nvpmodel -m 3 && echo "15W power-save mode"'
alias perf='sudo nvpmodel -q && sudo jetson_clocks --show 2>/dev/null | grep -E "(CPU|GPU|

# --- Monitoreo ---
alias stats='tegrastats --interval 1000'
alias temps='cat /sys/class/thermal/thermal_zone*/temp 2>/dev/null | awk "{printf \"%1f C'
alias memfree='free -h | grep -E "^(Mem|Swap)"'
alias gpucheck='tegrastats | head -1'

# --- Docker GPU ---
alias drun='docker run --runtime nvidia --gpus all'
alias ollama='docker exec -it ollama ollama'
```


~/ .bashrc — BLOQUE COMPLETO (parte 3 de 3)

```
# --- Sistema de archivos ---
alias ls='ls --color=auto'
alias ll='ls -aFh --color=auto'
alias la='ls -A --color=auto'
alias ..='cd ..'
alias ...='cd ../../'
alias mkdir='mkdir -pv'

# --- Sistema ---
alias update='sudo apt update && sudo apt upgrade -y && sudo apt autoremove -y'
alias disk='df -h | grep -E "^(/dev|Filesystem)"'
alias ports='ss -tulnp'

# --- Git ---
alias gs='git status'
alias gd='git diff'
alias gl='git log --oneline -10'

# --- Historial optimizado ---
HISTCONTROL=ignoreboth
HISTSIZE=100000
HISTFILESIZE=10000000
HISTIGNORE="ls:ll:cd:pwd:bg:fg:history"
shopt -s histappend
PROMPT_COMMAND="history -a; history -n; $PROMPT_COMMAND"

# --- Deduplicar PATH ---
export PATH=$(echo -n "$PATH" | awk -v RS=: -v ORS=: '!x[$0]++' | sed 's/:$/ /')

# --- Saludo ---
if command -v tegrastats &>/dev/null; then
    POWER_MODE=$(sudo nvpmode1 -q 2>/dev/null | grep "NV Power Mode" | awk '{print $NF}')
    echo "Jetson AGX Orin 64GB | Mode: ${POWER_MODE:-unknown} | $(free -h | grep Mem | awk
fi
```

22. Aplicar y verificar

Paso 1: Guardar y cerrar nano

Presiona Ctrl + O (guardar), luego Enter, luego Ctrl + X (salir).

Paso 2: Aplicar los cambios sin cerrar la terminal

Terminal

```
source ~/.bashrc
```

Paso 3: Verificar CUDA

Terminal

```
nvcc --version | grep "release"
# -> release 12.6, V12.6.68

echo $CUDA_HOME
# -> /usr/local/cuda-12.6
```

Paso 4: Verificar flags de compilacion

Terminal

```
echo "Build cores: $MAKEFLAGS"
# -> Build cores: -j12
```

Paso 5: Probar alias de monitoreo

Terminal

```
memfree
# -> Mem:          7.3Gi/61Gi
# -> Swap:         0B/31Gi
```

Paso 6: Probar modo MAXN

Terminal

```
maxn
# -> MAXN mode + clocks locked

perf
# -> NV Power Mode: MAXN
# -> CPU Freq: 2201600 ...
```

Paso 7: Verificar que no hay errores de sintaxis

Terminal

```
# Verificar sintaxis sin ejecutar:  
bash -n ~/.bashrc && echo "Sintaxis OK" || echo "Error de sintaxis"
```

23. Hugging Face Token (Opcional)

Si necesitas descargar modelos con restricciones (gated models) como Llama 3, Mistral, Gemma, etc., necesitas un token de Hugging Face.

Opcion A: Variable de entorno en .bashrc

```
~/ .bashrc

# Agregar DESPUES del bloque principal en .bashrc:
export HF_TOKEN="hf_XXXXXXXXXXXXXXXXXXXX"
```

Opcion B: CLI login (mas seguro)

```
Terminal

pip install huggingface_hub
huggingface-cli login
# El token se guarda en ~/.cache/huggingface/token
```

Para obtener tu token:

1. Ve a <https://huggingface.co/settings/tokens>
2. Crea un token de tipo "Read" (solo lectura es suficiente)
3. Copia el token y pegalo en uno de los metodos anteriores

IMPORTANTE

Nunca subas tu .bashrc a un repositorio publico si contiene tu HF_TOKEN. La Opcion B (CLI login) es mas segura porque almacena el token en un archivo separado que es mas facil de excluir con .gitignore.

24. Mover cache de HuggingFace a NVMe (Opcional)

Los modelos descargados de Hugging Face pueden ocupar 10-100 GB. Si tienes un disco NVMe montado (por ejemplo en /data), redirige la cache para tener mas espacio y mejor velocidad de lectura.

```
~/ .bashrc — Cache en NVMe

# Reemplazar la línea HF_HOME en tu .bashrc con:
export HF_HOME="/data/.cache/huggingface"
mkdir -p "$HF_HOME"
```

Verifica que el NVMe esta montado:

```
Terminal

df -h /data
# -> /dev/nvme0n1p1 932G 45G 840G 6% /data
```

25. Resolucion de problemas

Problema	Causa probable	Solucion
nvcc: not found despues de source	CUDA_HOME incorrecto o no apunta a cuda-12.6	Verificar: ls /usr/local/cuda-12.6/bin/nvcc
El saludo se queda colgado al abrir terminal	El comando sudo dentro de \$() pide contrasena	Eliminar las lineas del saludo o configurar sudo sin password para nvpmoel
Entradas duplicadas en PATH	Multiples source ~/.bashrc	El bloque de deduplicacion lo resuelve. Verificar con echo \$PATH
Alias ollama conflict a con instalacion nativa	Ollama instalado nativamente y tambien en Docker	Eliminar la linea del alias ollama del .bashrc
Terminal se abre muy lenta (3+ segundos)	NVM cargando al inicio	Usar la version con carga diferida (lazy loading) de la seccion 11
pip: command not found	~/.local/bin no esta en PATH	Verificar que la linea PATH="\$HOME/.local/bin" esta en .bashrc
Error de sintaxis al hacer source	Caracter mal pegado o comillas faltantes	Ejecutar: bash -n ~/.bashrc para encontrar la linea con error. Restaurar backup si es necesario.

26. Fuentes y referencias

1. NVIDIA Developer Forums — nvcc not found JetPack 6 — <https://forums.developer.nvidia.com/t/nvcc-not-found-jetpack-6/296512>
2. NVIDIA Tegrastats Utility Documentation (JetPack 6.2) — <https://docs.nvidia.com/jetson/archives/r36.2/DeveloperGuide/AT/JetsonLinuxDevelopmentTools/TegrastatsUtility.html>
3. NVIDIA Developer Forums — TensorRT include and lib paths — <https://forums.developer.nvidia.com/t/i-want-to-know-the-include-and-lib-path-of-tensorrt-installed-by-jetpack/278017>
4. Hugging Face — Environment Variables Reference — https://huggingface.co/docs/huggingface_hub/en/package_reference/environment_variables
5. Hugging Face GitHub — Enabling hf_transfer by default (Issue #2279) — https://github.com/huggingface/huggingface_hub/issues/2279
6. Ollama FAQ — Setting Environment Variables on Linux — <https://docs.ollama.com/faq>
7. Ollama GitHub — Global Configuration Variables (Issue #2941) — <https://github.com/ollama/ollama/issues/2941>
8. DigitalOcean — What Is .bashrc in Linux? Configuration Explained — <https://www.digitalocean.com/community/tutorials/bashrc-file-in-linux>
9. Meta Redux — Supercharge Your Bash History — <https://metaredux.com/posts/2020/07/07/supercharge-your-bash-history.html>
10. Nulogy Engineering — Lazy Load NVM for Faster Shell Start — <https://til-engineering.nulogy.com/Slow-Terminal-Startup-Tip-Lazy-Load-NVM/>
11. OneUptime — How to Customize Your Bash Prompt (PS1) on Ubuntu — <https://oneuptime.com/blog/post/2026-03-02-how-to-customize-your-bash-prompt-ps1-on-ubuntu/view>
12. pyenv GitHub — Set up shell environment for pyenv — <https://github.com/pyenv/pyenv>
13. NVIDIA JetsonHacks — NVPModel for Jetson AGX — <https://jetsonhacks.com/2018/10/07/nvpmodel-nvidia-jetson-agx-xavier-developer-kit/>
14. RidgeRun Developer Wiki — Jetson Orin Evaluating Performance — https://developer.ridgerun.com/wiki/index.php/NVIDIA_Jetson_Orin/JetPack_5.0.2/Performance_Tuning/Evaluating_Performance
15. NVIDIA Forums — Docker Runtime on JetPack 6.2 — <https://forums.developer.nvidia.com/t/nvidia-runtime-components-not-detected-in-docker-container-on-jetson-orin-nx-jetpack-6-2/324955>